

Reflection

Factor II: Dependencies

Dependencies were managed using virtual environments and a requirements.txt file. Using a virtual environment ensured that all project dependencies remained isolated from the global Python installation, preventing conflicts with other projects running on the same system. The requirements.txt file was used to store all installed libraries and their versions, making it easy to recreate the same development environment on different machines. This approach improved consistency, portability, and compatibility across development and deployment environments.

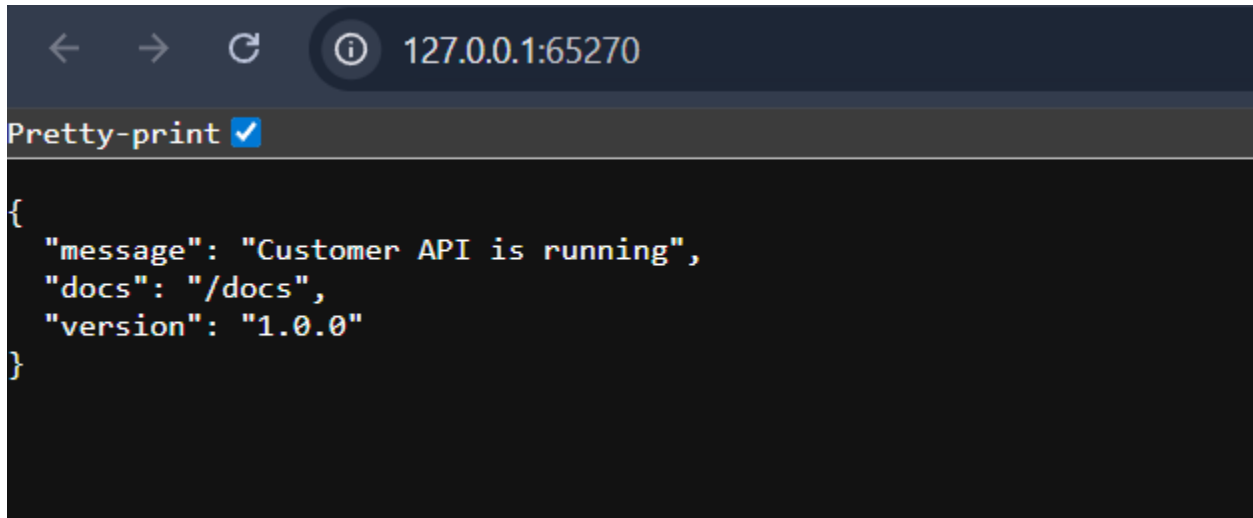
Factor IV: Backing Services

The project followed the Backing Services principle by using SQLAlchemy ORM to separate the application logic from the database engine. Instead of tightly coupling the application to PostgreSQL-specific queries, SQLAlchemy provided an abstraction layer that allowed database interactions using Python objects. Because of this design, the application could later switch from PostgreSQL to other database systems such as MySQL, SQLite, or MariaDB without requiring major changes to the core business logic. This increased the flexibility and scalability of the application architecture.

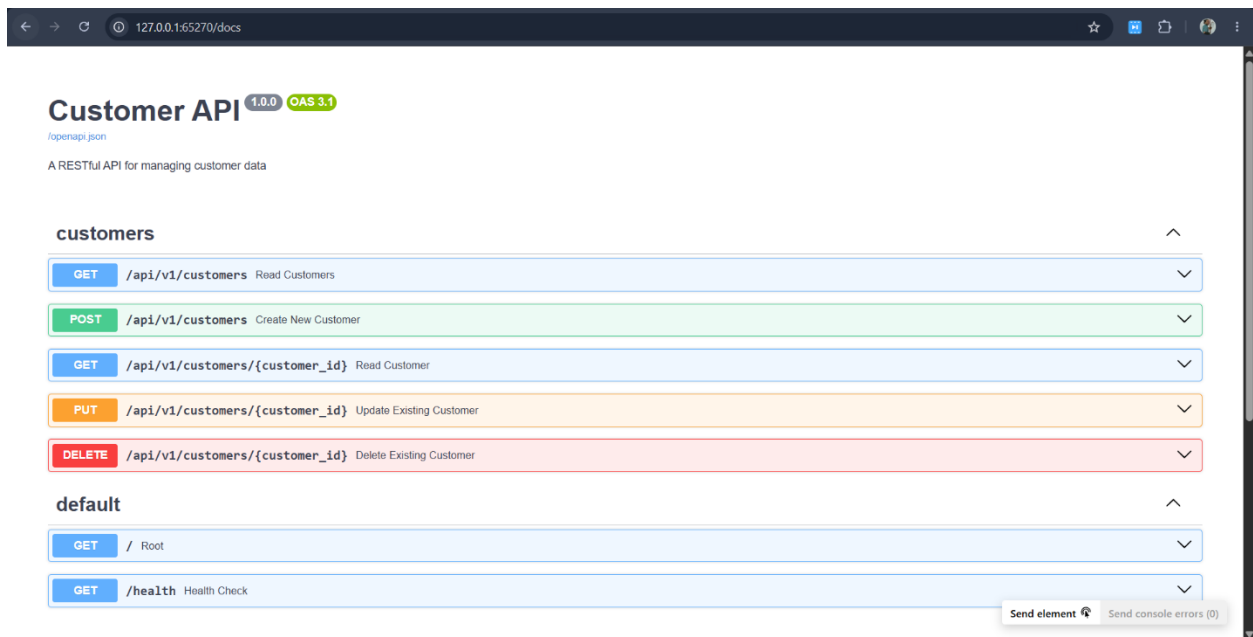
Factor III: Config Management

Sensitive configuration values were managed using environment variables stored in a .env file. Database credentials and other environment-specific settings were not hardcoded into the source code, improving the overall security of the application. This approach also made the system more flexible because different environments, such as development, testing, and production, could use separate configuration values without modifying the application code. As a result, deployment and environment management became easier, safer, and more maintainable.

Visiting the local host



Swagger UI Docs



GET All Customers Request

customers

GET/api/v1/customersRead Customers

Retrieve multiple customers with pagination

Parameters

Name

Description

skip

integer

(query)

0

limit

integer

(query)

100

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://127.0.0.1:65270/api/v1/customers?skip=0&limit=100' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:65270/api/v1/customers?skip=0&limit=100

Server response

Request URL

http://127.0.0.1:65270/api/v1/customers?skip=0&limit=100

Server response

Code

Details

200

Response body

```
{
  "customerNumber": 101,
  "customerName": "Atelier graphique",
  "contactLastName": "Schmitt",
  "contactFirstName": "Carine",
  "phone": "46.32.2255",
  "addressLine1": "34, rue Royale",
  "addressLine2": null,
  "city": "Nantes",
  "state": null,
  "postalCode": "44000",
  "country": "France",
  "salesRepEmployeeNumber": 1370,
  "creditLimit": "21000.00"
},
{
  "customerNumber": 112,
  "customerName": "Signal Gift Stores",
  "contactLastName": "King",
  "contactFirstName": "Jean",
  "phone": "7825551838",
  "addressLine1": "6490 Strong St.",
  "addressLine2": null,
  "city": "Las Vegas",
  "state": "NV",
  "postalCode": "89030",
  "country": "USA",
  "salesRepEmployeeNumber": 1166,
  "creditLimit": "21000.00"
}
```

Response headers

```
content-length: 32355
content-type: application/json
date: Sun, 10 May 2020 14:56:17 GMT
server: wicorn
```

Responses

Code

Description

200

Successful Response

[illegible]

I checked customer number 103

GET

/api/v1/customers/{customer_id}

Read Customer

Retrieve a single customer by ID

Parameters

Cancel

Name	Description
customer_id * required	
integer (path)	103

Execute

Clear

Responses

curl

```
curl -X 'GET' \
  https://127.0.0.1:55270/api/v1/customers/103/ \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:55270/api/v1/customers/103
```

Server response

Code	Details
200	Response body <pre>{ "customerNumber": 103, "customerName": "Atelier graphique", "contactLastName": "Chet", "contactFirstName": "Carine ", "phone": "06.38.2055", "addressLine1": "54, rue Royale", "addressLine2": null, "city": "Nantes", "state": null, "postalCode": "44000", "country": "France", "salesrepEmployeeNumber": 1370, "creditLimit": 770000 }</pre>

Similarly, I checked customer number 99999 which was not in the database, and got the error as assumed.

GET

/api/v1/customers/{customer_id}

Read Customer

Retrieve a single customer by ID

Parameters

Cancel

Name	Description
customer_id * required	
integer	9999999
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:65270/api/v1/customers/9999999' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:65270/api/v1/customers/9999999
```

Server response

Code	Details
404	Error: Not Found

Undocumented

Response body

```
{
  "detail": "Customer not found"
}
```

Download

Response headers

```
content-length: 31
content-type: application/json
date: Sun, 10 May 2026 15:10:11 GMT
server: uvicorn
```